



创建 DataFrame 的常用方法

详细介绍创建 DataFrame 的常用方法，并逐一演示。

1. 从字典创建

代码块

```
1  import pandas as pd
2
3  # 方法1: 从字典创建 (最常用)
4  data = {
5      '姓名': ['张三', '李四', '王五'],
6      '年龄': [25, 30, 28],
7      '城市': ['北京', '上海', '广州']
8  }
9  df1 = pd.DataFrame(data)
10 print("方法1 - 从字典创建:")
11 print(df1)
12 print()
```

2. 从列表/数组创建

代码块

```
1  # 方法2: 从嵌套列表创建
2  data_list = [
3      ['张三', 25, '北京'],
4      ['李四', 30, '上海'],
5      ['王五', 28, '广州']
6  ]
7  df2 = pd.DataFrame(data_list, columns=['姓名', '年龄', '城市'])
8  print("方法2 - 从列表创建:")
9  print(df2)
10 print()
```

3. 从 NumPy 数组创建

代码块

```
1  import numpy as np
2
3  # 方法3: 从 NumPy 数组创建
4  array_data = np.array([
5      [25, 70, 175],
6      [30, 65, 180],
7      [28, 72, 168]
8  ])
9  df3 = pd.DataFrame(array_data, columns=['年龄', '体重', '身高'])
10 print("方法3 - 从NumPy数组创建:")
11 print(df3)
12 print()
```

4. 从 CSV 文件创建

代码块

```
1  # 方法4: 从CSV文件读取 (需要先有csv文件)
2  # 先创建一个示例CSV文件
3  df_temp = pd.DataFrame({
4      '产品': ['苹果', '香蕉', '橙子'],
5      '价格': [5.5, 3.2, 4.8],
6      '数量': [100, 150, 80]
7  })
8  df_temp.to_csv('sample.csv', index=False, encoding='utf-8-sig')
9
10 # 从CSV文件读取
11 df4 = pd.read_csv('sample.csv', encoding='utf-8-sig')
12 print("方法4 - 从CSV文件创建:")
13 print(df4)
14 print()
```

5. 从 Excel 文件创建

代码块

```
1  # 方法5: 从Excel文件读取
2  # 先创建一个示例Excel文件
3  df_temp.to_excel('sample.xlsx', index=False)
4
5  # 从Excel文件读取
6  df5 = pd.read_excel('sample.xlsx')
7  print("方法5 - 从Excel文件创建:")
8  print(df5)
9  print()
```

6. 从 Series 创建

代码块

```
1  # 方法6: 从Series对象创建
2  s1 = pd.Series([1, 2, 3], name='A列')
3  s2 = pd.Series([4, 5, 6], name='B列')
4  df6 = pd.DataFrame({'A列': s1, 'B列': s2})
5  print("方法6 - 从Series创建:")
6  print(df6)
7  print()
```

7. 从数据库查询创建

代码块

```
1  # 方法7: 从SQL数据库查询 (需要数据库连接)
2  # 这里展示代码结构, 实际使用需要真实数据库
3  """
4  import sqlite3
5  conn = sqlite3.connect('example.db')
6  query = "SELECT * FROM users"
7  df7 = pd.read_sql_query(query, conn)
8  conn.close()
```

```
9     """
10     print("方法7 - 从数据库查询创建:")
11     print("(需要实际数据库连接)")
12     print()
```

8. 从 JSON 数据创建

代码块

```
1     # 方法8: 从JSON数据创建
2     import json
3
4     json_data = '''
5     [
6         {"姓名": "张三", "年龄": 25, "城市": "北京"},
7         {"姓名": "李四", "年龄": 30, "城市": "上海"},
8         {"姓名": "王五", "年龄": 28, "城市": "广州"}
9     ]
10    '''
11    data_dict = json.loads(json_data)
12    df8 = pd.DataFrame(data_dict)
13    print("方法8 - 从JSON创建:")
14    print(df8)
15    print()
```

9. 使用字典列表创建

代码块

```
1     # 方法9: 从字典列表创建
2     records = [
3         {'姓名': '张三', '年龄': 25, '城市': '北京'},
4         {'姓名': '李四', '年龄': 30, '城市': '上海'},
5         {'姓名': '王五', '年龄': 28, '城市': '广州'}
6     ]
7     df9 = pd.DataFrame(records)
8     print("方法9 - 从字典列表创建:")
9     print(df9)
```

```
10 print()
```

10. 创建空 DataFrame

代码块

```
1  # 方法10: 创建空DataFrame后填充
2  df10 = pd.DataFrame(columns=['姓名', '年龄', '城市'])
3  # 可以后续添加数据
4  df10.loc[0] = ['张三', 25, '北京']
5  df10.loc[1] = ['李四', 30, '上海']
6  print("方法10 - 创建空DataFrame后填充:")
7  print(df10)
8  print()
```

完整可运行示例

代码块

```
1  import pandas as pd
2  import numpy as np
3
4  print("="*50)
5  print("DataFrame 创建方法演示")
6  print("="*50)
7
8  # 1. 字典创建
9  df_dict = pd.DataFrame({
10      '姓名': ['张三', '李四', '王五'],
11      '年龄': [25, 30, 28],
12      '城市': ['北京', '上海', '广州']
13  })
14  print("\n1. 从字典创建:")
15  print(df_dict)
16
17  # 2. 列表创建
18  df_list = pd.DataFrame(
```

```
19     [['张三', 25, '北京'], ['李四', 30, '上海']],
20     columns=['姓名', '年龄', '城市']
21 )
22 print("\n2. 从列表创建:")
23 print(df_list)
24
25 # 3. NumPy数组创建
26 df_array = pd.DataFrame(
27     np.random.randint(0, 100, (3, 3)),
28     columns=['A', 'B', 'C']
29 )
30 print("\n3. 从NumPy数组创建:")
31 print(df_array)
32
33 # 4. 字典列表创建
34 df_records = pd.DataFrame([
35     {'姓名': '张三', '年龄': 25},
36     {'姓名': '李四', '年龄': 30}
37 ])
38 print("\n4. 从字典列表创建:")
39 print(df_records)
40
41 print("\n" + "="*50)
42 print("所有方法演示完成!")
```



方法对比总结

方法	适用场景	优点	缺点
字典	小规模数据、手动创建	直观、易读	大数据量不便
列表	已有结构化列表数据	简单直接	需要指定列名
NumPy数组	数值计算、矩阵运算	性能好	只适合数值型
CSV文件	外部数据导入	通用性强	需要文件存在
Excel文件	Excel数据导入	保留格式	需要openpyxl库
Series	已有Series对象	灵活组合	步骤较多

数据库	数据库数据提取	实时数据	需要数据库连接
JSON	API数据、Web数据	网络友好	需要解析
字典列表	API返回数据	结构清晰	内存占用较大
空DataFrame	动态添加数据	灵活	性能较差

最佳实践建议

1. 小数据手动创建 → 使用字典或列表
2. 外部数据导入 → 使用 `read_csv()` 或 `read_excel()`
3. API数据 → 使用 JSON 或字典列表
4. 大规模数值计算 → 使用 NumPy 数组
5. 数据库数据 → 使用 `read_sql_query()`

根据您的具体需求选择最合适的方法！